

REGISTRATION NO: H250576C

FULL NAME: Rufaro Chelsea Zidenga

DEPARTMENT: Computer Science

ASSESSMENT: Lab Practicals Report

YEAR: 2026

MODULE: Object Oriented Programming in Java

LAB PRACTICALS OVERVIEW

I successfully executed and completed all eleven lab sessions outlined in the module manual, ensuring every code snippet compiled cleanly and produced the intended output.

Lab 1: Classes and Objects

Objectives

- Construct Java classes containing attributes (state) and methods (behavior).
- Instantiate runtime objects using the new operator and manipulate members via dot notation.
- Understand reference variables as heap memory addresses rather than direct object values.
- Identify null states and write defensive code to avoid runtime NullPointerException errors.

Work Done

Developed a Student class with attributes for name, roll number, and GPA, alongside helper methods display() and isHonours(). Created a ReferenceDemo program to demonstrate variable aliasing and memory address comparison using == versus value comparison. Furthermore, built a Course class containing an embedded Student reference field.

Results

The console output correctly rendered student profiles, marking Tariro (GPA 3.8) as an honors candidate while Farai (GPA 3.1) was not. Modifying attributes via an alias reference updated the original instance directly (confirming a == b evaluated to true, whereas a == c evaluated to false). Unassigned references were properly validated before invocation.

Conclusion

Classes serve as structural templates, whereas objects represent instantiated entities stored on the heap. Because reference variables store pointer addresses, variable assignment duplicates the memory reference rather than cloning the underlying object. Explicit null checks are critical to maintaining program stability.

Lab 2: Constructors, the this Keyword and static Members

Objectives

- Initialize object attributes securely through custom constructors.
- Implement constructor overloading and eliminate code duplication using explicit constructor chaining via this(...).

- Utilize the this self-reference keyword to disambiguate class attributes from local parameters.
- Differentiate between instance-level members and class-level (static) members.
- Analyze execution precedence across static and instance initialization blocks.

Work Done

Designed a Book class featuring three overloaded constructors linked via this(...) chaining, supported by a static counter tracking instance allocation and both static and instance initialization blocks. Programmed an InitOrder class to observe class loading sequences and constructed a Point class featuring a copy constructor.

Results

Instantiated book records were assigned sequential IDs (1 through 3), and Book.getCount() verified a total count of 3. The static initialization block executed exactly once upon class load, whereas instance blocks executed prior to each constructor invocation. The copy constructor successfully generated a distinct object instance holding identical coordinates.

Conclusion

Constructors establish mandatory initial states for objects, and chaining streamlines setup logic across multiple constructors. Static components belong to the class blueprint and remain shared across all instances, whereas instance variables are unique to each object. Class initialization strictly adheres to a set sequence: static elements, instance blocks, and finally constructor execution.

Lab 3: Encapsulation and Access Modifiers

Objectives

- Restrict direct field access using private scope modifiers while providing controlled exposure via getters and setters.
- Apply Java's four access levels (private, package-private, protected, public) appropriately.
- Incorporate state validation checks within mutators and constructors to maintain class invariants.
- Design robust domain classes that prevent illegal or inconsistent state mutations.

Work Done

Created a secure BankAccount class with private fields, defensive constructor checks, and strictly encapsulated balance updates allowed only through deposit() and withdraw() methods. Designed a Team class to demonstrate the risks of exposing mutable internal lists versus

returning read-only wrappers.

Results

Depositing $250 raised the balance to $750.00, while a valid $100 withdrawal succeeded and an excessive $5000 request was safely rejected without altering the account balance. Invalid inputs like negative deposits triggered exceptions. Attempting to modify an unmodifiable list view generated an UnsupportedOperationException.

Conclusion

Data hiding combined with validated mutators guarantees object integrity and enables internal refactoring without breaking dependent code. Exposing mutable internal references directly compromises encapsulation; mutators must always return defensive copies or unmodifiable abstractions.

Lab 4: Inheritance

Objectives

- Derive specialized classes using the extends keyword to reuse base class features.
- Invoke superclass constructors and overridden parent methods using super.
- Apply protected, final, and the @Override annotation effectively.
- Understand superclass constructor invocation chains and the universal parent class java.lang.Object.
- Evaluate structural decisions between class inheritance ("is-a") and object composition ("has-a").

Work Done

Implemented an Employee base class alongside specialized Manager and Developer subclasses, overriding salary calculations. Traced constructor execution across a multi-tier hierarchy (A → B → C), and created a Car class containing an internal Engine instance to illustrate composition.

Results

Console output confirmed parent constructors execute prior to child constructors. Calling describe() yielded $2000.00 for standard employees, $4300.00 for managers, and $3250.00 for developers, reflecting specialized compensation logic. The composition pattern successfully delegated engine start functionality.

Conclusion

Inheritance enables code reuse and domain specialization for genuine "is-a" relationships, ensuring parent initialization occurs first. Annotating overrides prevents compile-time method signature mismatches, whereas composition provides greater flexibility for component-based "has-a" designs.

Lab 5: Polymorphism

Objectives

- Distinguish static polymorphism (method overloading) from dynamic polymorphism (method overriding).
- Utilize superclass references to manage subclass instances through dynamic method dispatch.
- Perform safe type downcasting using instanceof type matching.
- Write extensible architectures that accommodate new child types seamlessly.

Work Done

Overloaded the add method in a Calculator utility class across various parameter signatures. Formulated an abstract Shape foundation with Circle, Rectangle, and Triangle implementations evaluated within a single polymorphic array. Investigated non-polymorphic field shadowing and static method hiding using Animal and Dog classes.

Results

Overloaded method calls resolved correctly based on argument types during compilation. The shape iteration dynamically computed areas (78.54, 24.00, and 12.00, totaling 114.54) without needing explicit concrete type checks. Non-overridden member variables and static methods bound strictly to declared reference types rather than runtime instances.

Conclusion

Method overloading is resolved statically by the compiler, while overriding is dynamically resolved by the JVM at runtime based on actual heap instances. Because polymorphism applies exclusively to instance methods, fields and static calls depend on declared variable types. Polymorphic design ensures code extensibility without refactoring existing caller logic.

Lab 6: Abstraction, Abstract Classes and Interfaces

Objectives

- Define abstract classes and methods to establish incomplete specifications that enforce

subclass concrete contracts.
- Construct interfaces featuring abstract, default, static, and private capability methods.
- Select appropriate design boundaries between abstract classes and interfaces.
- Apply the "program to an interface, not an implementation" design principle.
- Incorporate standard library interfaces such as Comparable for object ordering.

Work Done

Built an abstract Vehicle base class extended by concrete Truck and Motorbike types. Defined Payable and Loggable interfaces featuring default and static utility routines, implemented concurrently by CardPayment and CashPayment classes. Enhanced a Player class with Comparable to support score-based sorting.

Results

A unified fleet loop calculated trip costs ($80.00 for trucks, $15.00 for motorbikes per 100 km). CardPayment leveraged default logging while CashPayment supplied custom receipt logic, blocking transactions exceeding financial thresholds. Player arrays auto-sorted in descending order of performance score.

Conclusion

Abstract classes are ideal for tightly coupled hierarchies sharing state and common logic, whereas interfaces define flexible contracts that can be shared across unrelated classes. Designing against interface contracts yields adaptable code bases capable of integrating standard framework utilities seamlessly.

Lab 7: Exception Handling

Objectives

- Analyze the Java exception hierarchy and differentiate checked exceptions from unchecked runtime errors.
- Construct structured error recovery blocks using try, catch, finally, throw, and throws.
- Develop custom domain exceptions carrying detailed context payload data.
- Automate resource cleanup using the try-with-resources statement.
- Ensure predictable method failure handling to preserve system invariants.

Work Done

Configured structured catch blocks and multi-catch routines handling arithmetic errors, array indexing violations, and null references. Created a custom checked InsufficientFundsException for a Wallet domain model, and implemented an AutoCloseable database Connection class to

verify automatic resource disposal and exception chaining.

Results

The finally block executed reliably across all pass/fail branches. The Wallet rejected transactions exceeding available balances, providing precise deficit figures. Managed connection resources closed in reverse initialization order even when runtime queries failed, maintaining complete root cause traces.

Conclusion

Exception handling isolates normal execution paths from error routines, ensuring failures are safely handled. Utilizing tailored exceptions, structured catch hierarchies, and automatic resource management prevents resource leaks and keeps systems in stable states.

Lab 8: Generics and the Collections Framework

Objectives

- Implement generic classes, interfaces, and methods utilizing bounded type parameters.
- Understand type erasure and primitive type wrapping via autoboxing.
- Select optimal data structures across List, Set, Map, and Queue implementations.
- Sort and search datasets using Comparable, Comparator, and Collections algorithms.
- Enforce interface-based declarations (e.g., List<String> list = new ArrayList<>()).

Work Done

Constructed generic Box<T> and Pair<K, V> wrappers alongside bounded generic utility functions. Experimented with ArrayList, TreeSet, HashMap, TreeMap, and ArrayDeque to evaluate sequence handling, uniqueness constraints, key-value lookups, word frequency parsing, and LIFO/FIFO queue mechanics. Evaluated object sorting via Comparable and custom Comparator instances.

Results

Generic type declarations caught compile-time type mismatches, eliminating explicit casting. Lists preserved duplicate elements whereas Sets automatically deduplicated items. Maps managed key updates efficiently, and word counting produced accurate frequency distributions. Stack operations executed LIFO order while Queues processed items FIFO.

Conclusion

Generics ensure strong type safety without sacrificing code reusability. Choosing the correct collection structure—List for indexed order, Set for uniqueness, Map for key-based indexing,

and Queue for processing sequences—is crucial for application efficiency. Comparators provide dynamic sorting capabilities alongside natural object ordering.

Lab 9: Nested Types, Enums, Records and the Object Contract

Objectives

- Distinguish between static nested, inner member, local, and anonymous classes.
- Define fixed constant enumerations using enum structures containing fields, constructors, and behavior.
- Model immutable data containers efficiently using Java record constructs.
- Implement cohesive equals, hashCode, and toString method overrides.

Work Done

Constructed a unified application featuring all four nested class types, including an inner iterator accessing private outer state. Implemented Day, Planet, and Operation enumerations featuring constant-specific behavior. Built a custom Point class overriding value equality rules alongside an immutable Coordinate record containing validation logic.

Results

Inner class instances successfully traversed private enclosing data structures. Enumerations evaluated seamlessly within switch expressions and mathematical operations. Equivalent Point instances evaluated as unequal by reference (!=) but equal by value (equals()), maintaining single-entry integrity within hash-based collections. Records streamlined coordinate calculations effortlessly.

Conclusion

Nested types logically group helper classes close to their usage context. Enums offer robust, type-safe constant definitions, while records eliminate boilerplates for immutable data carriers. Ensuring equals and hashCode are overridden together is mandatory for correct behavior in hash-based data structures.

Lab 10: File I/O, Lambdas and Streams

Objectives

- Perform text file operations via java.nio.file while managing checked IOException events.

- Serialize and deserialize domain objects using structured CSV formatting.
- Write functional lambda expressions, method references, and custom functional interfaces.
- Process collections declaratively using Stream API operations (filtering, mapping, sorting, reducing).

Work Done

Created, appended, and read data files via students.txt, incorporating graceful handling for missing files. Designed a StudentRepository to save and restore Learner records in CSV format. Applied Predicate, Function, Consumer, and Supplier functional interfaces, executing declarative processing pipelines over employee datasets.

Results

Dataset operations completed cleanly, with non-existent file lookups failing safely without crashing. Serialized entities reloaded into memory with full data fidelity. Stream pipelines filtered high-earning staff, computed organizational metrics ($19,400 total, $3,880.0 average), and grouped records by department efficiently.

Conclusion

The modern java.nio.file package paired with robust exception handling guarantees safe file persistence. Stream operations combined with lambda expressions provide clean, highly readable data transformation pipelines without mutating source collections.

Lab 11: Design Patterns and the Mini-Project

Objectives

- Implement core architectural patterns (Singleton, Factory, Strategy, Observer) to resolve common software problems.
- Apply SOLID principles emphasizing modular design, extensible architectures, and interface dependencies.
- Integrate OOP concepts, custom exceptions, collection structures, file persistence, and stream pipelines into an integrated application.

Work Done

Engineered a thread-safe Singleton AppConfig, a NotificationFactory instantiating messaging channels, a Checkout engine accepting dynamic discount strategies, and a GradeBook publishing events to subscribed listeners. Designed the architecture for a Library Management System utilizing abstract items, borrowable interfaces, custom domain exceptions, map storage,

CSV persistence, and stream-based queries.

Results

Singleton calls verified identical instance references across calls. The notification factory returned correct operational instances based on input descriptors. Strategy pattern dynamic swapping adjusted price calculations smoothly, while Observer listeners received event alerts immediately upon grade updates.

Conclusion

Design patterns offer standardized solutions to recurring architectural challenges: Singleton governs instance control, Factory hides instantiation details, Strategy enables runtime algorithm swapping, and Observer achieves loose coupling. The mini-project demonstrates how key Java language features work together in enterprise application design.

Overall Conclusion

Across the eleven laboratory practicals, my implementation progressed from basic single-class setups to an integrated multi-tier application, fulfilling all core course objectives:

- **Labs 1–3:** Mastered class modeling, object instantiation, state management, and encapsulation mechanisms.
- **Labs 4–6:** Applied object-oriented abstraction, class inheritance, dynamic polymorphism, and interface contracts.
- **Labs 7–10:** Enhanced application stability and data processing capabilities through exception handling, generic types, collection frameworks, file I/O operations, lambdas, and streams.
- **Lab 11:** Combined structural patterns and core concepts into a cohesive mini-project architecture.

The primary takeaway is the importance of keeping data structures private, depending on flexible interface abstractions, favoring composition over inheritance, and managing errors deliberately. These practices lead to robust, maintainable, and scalable Java software systems.

Note: *Artificial intelligence tooling was utilized as a supportive study aid throughout selected practical exercises.*

Among all exercises, I found the **Bubble Sort algorithm implementation** to be the most practical and insightful, as it provided fundamental experience with dataset sorting mechanics widely applied in real-world software systems.